广电用户账单与订单数据查询进阶

- 统计订单的消费类型
- 统计用户每年消费应付总额
- 统计用户每月消费应付总额
- 统计用户每月实际账单金额
- 查询用户宽带订单的地址数据
- 抽样统计用户订购产品情况

一. 统计订单的消费类型

- 掌握 Hive 的常见内置函数
- 使用条件函数及类型转换函数对订单的消费类型进行分类

Hive 内置函数

1. Hive 内置函数介绍

Hive 将一些常用功能以内置函数的方式实现，以此简化查询语句，提升查询效率。常见的 Hive 内置函数种类如下。

- 数学函数
- 字符函数
- 条件函数
- 转换函数
- 日期函数
- 聚合函数

2. Hive 内置函数查看

查看 Hive 提供的内置函数。

```
SHOW FUNCTIONS;
```

查看某个函数具体的使用方法。

```
DESC FUNCTION EXTENDED 函数名;
```

比如：

```
DESC FUNCTION EXTENDED not;
```

条件函数

1. 条件函数的语法

如果查询过程需要根据不同的条件求出不同的结果，可以在查询过程中引入条件函数。

条件函数的语法

```
CASE A                      -- A：数据表中的一个字段，对其值进行判断
WHEN B THEN C               -- 若 A 的值为 B，则返回 C
[WHEN D THEN E]*            -- 可选，可继续添加 0 个或多个判断分支；若 A 的值为 D，则返回 E
[ELSE F]                    -- 可选，所有分支条件均不成立时返回 F
END                         -- 结束符
```

> *B、D 的数据类型必须与字段 A 保持完全一致，否则会导致语法错误。*

按 mode_time（主销售品标识）对订单分类，判断结果作为查询结果的一列输出。

```
SELECT phone_no, mode_time,
CASE mode_time
WHEN 'Y' THEN '主销售产品'
WHEN 'N' THEN '附销售产品'
ELSE '无效标识'
END
FROM order_index;
```

mode_time 为 'Y' 时返回“主销售产品”，为 'N' 时返回“附销售产品”，其余值返回“无效标识”。

2. 条件函数与列别名结合使用

在上一条语句的基础上为条件函数添加了列别名，使结果列名显示为“产品标识”。

```
SELECT phone_no, mode_time,
CASE mode_time
WHEN 'Y' THEN '主销售产品'
WHEN 'N' THEN '附销售产品'
ELSE '无效标识'
END AS `产品标识`
FROM order_index;
```

- 条件函数的执行结果作为 SELECT 语句查询结果的一列，可以被视为一个整体，因此可以为条件函数添加列别名。

- 列别名添加在条件函数 END 关键字之后。
- 以“[AS] 列别名”的方式添加。
- AS 关键字与普通设置列别名的过程一样，为可省略部分。
- 中文列别名需要使用反单引号（`）引起来。

类型转换函数

在 Hive 中对不同类型的两个数据进行比较等操作时，Hive 会适时地对数据进行隐式的类型转换，在隐式转换过程中将使用 CAST 转换函数。

也可以直接使用 CAST 函数对指定的数据显式地进行类型转换。

类型转换函数的语法

```
CAST(VALUE AS TYPE)
```

- VALUE：待转换的值，可以是常量，也可以是数据表中的某个字段。
- TYPE：Hive 中的数据类型，用于指定 VALUE 值将要转换的数据类型。

```
SELECT CAST(99 AS DOUBLE);
```

将常量 99 显式转换为 DOUBLE 类型。

任务描述

使用条件函数及类型转换函数对订单的消费类型进行分类。

- 若订单金额大于等于 2000.0，则将这笔订单的消费类型认定为“高消费”；
- 若订单金额大于等于 1000.0 且小于 2000.0，则将消费类型认定为“中等消费”；
- 若订单金额小于 1000.0，则将消费类型认定为“低消费”。
- 对查询结果的消费类型列需要指定一个列名 type。
- 最终查询结果按照订单金额降序排列。
- 为了保证查询结果的有效性，还需对查询结果去除重复数据。

任务实现思路

- 使用 DISTINCT 关键字进行去重。
- order_index 数据表中的 cost 字段数据的类型为 STRING 类型，在查询结果的展示中可以使用 CAST 函数显式地将该字段的数据转换为浮点数。
- 使用 CASE…WHEN…THEN 条件函数对订单金额进行判断，可以设置以下 3 个分支条件。
    - 若订单金额大于等于 2000.0 为 TRUE，则输出“高消费”。

    - 若订单金额在 [1000.0, 2000.0) 区间为 TRUE，则输出“中等消费”。
    - 其他情况则输出“低消费”。
- 使用 ORDER BY 进行排序。

任务实现

统计订单的消费类型。

```
SELECT DISTINCT phone_no, orderno, prodprcname, CAST(cost AS FLOAT) AS cost_f,
CASE
WHEN cost >= 2000.0 THEN '高消费'
WHEN cost >= 1000.0 AND cost < 2000.0 THEN '中等消费'
ELSE '低消费'
END AS type
FROM order_index
ORDER BY cost_f DESC;
```

先用 CAST(cost AS FLOAT) 把字符串金额转成浮点数并命名为 cost_f，再用 CASE 把金额划分成“高消费 / 中等消费 / 低消费”三档（列别名 type），最后按 cost_f 降序排列；DISTINCT 保证整行结果不重复。

二. 统计用户每年消费应付总额

- 掌握 Hive 中的字符函数
- 使用字符函数对账单数据表 mmconsume_billevents 中的数据进行统计，计算每个用户的年度消费应付总额

字符函数

1. 大小写转换函数

LOWER 函数可以用于将字符串中的字符都转换成小写字符。

- 字符串中的小写字符维持原状。
- 只将大写字符转换成小写字符。

相应地，可以使用 UPPER 函数将字符串中的字符都转换成大写字符。

语法结构

```
LOWER(STRING S)
```

- S：可以是常量，也可以是数据表的某个字段。

示例

```
select Lower("Hello World");
```

2. 字符串长度函数

LENGTH 函数可以用于获取字符串的长度，返回结果是 S 包含的字符个数。

语法结构

```
LENGTH(STRING S)
```

- S：可以是常量，也可以是数据表的某个字段。

示例

```
select LENGTH("Hello World");
```

3. 字符串拼接函数

CONCAT 函数可以用于将两个字符串拼接起来形成一个新的字符串。

语法结构

```
CONCAT(STRING S1, STRING S2)
```

- S1、S2 是两个字符串。
- 均可以是常量，也可以是数据表的某个字段。

示例

```
select concat("Hello", "World");
```

4. 字符串截取函数

SUBSTR 函数可以用于从指定字符串中截取部分字符。

语法结构

```
SUBSTR(STRING S, INT START, INT LEN)
```

- S：代表被截取的字符串，可以是常量，也可以是数据表的某个字段。
- START 是整数，代表从第几位字符开始截取。
- LEN 同样是整数，代表共截取几个字符。
- 若省略 LEN，代表从第 START 位开始截取至字符串的最后一位。

示例

```
select substr("HelloWorld", 4,3);
```

5. 去除空格函数

TRIM 函数可以用于将字符串前后出现的空格全部去掉。

语法结构

```
TRIM(STRING S)
```

- S：需要去除空格的字符串，可以是常量，也可以是数据表的某个字段。

示例

```
select TRIM("   Hello World    ");
```

任务描述

使用字符函数对账单数据表 mmconsume_billevents 中的数据进行统计，计算每个用户的年度消费应付总额。

任务实现思路

- 使用字符截取函数 SUBSTR 对 year_month 字段值进行截取，截取该字段值的前 4 个字符，得到具体的年度账单。
- 结合 GROUP BY 关键字、SUM 函数，按照用户编号及年度账单进行分组，计算每个用户每年消费应付总额。

任务实现

先查看单个用户的账单明细，确认 year_month 字段的格式。

```
SELECT phone_no, year_month, should_pay
FROM mmconsume_billevents
WHERE phone_no = '5356294';
```

统计用户每年消费应付总额。

```
SELECT phone_no, SUBSTR(year_month, 1, 4), SUM(should_pay)
FROM mmconsume_billevents
GROUP BY phone_no, SUBSTR(year_month, 1, 4);
```

SUBSTR(year_month, 1, 4) 从账单年月中截取出年份，GROUP BY 按“用户编号 + 年份”分组后，再用 SUM(should_pay) 汇总出每个用户每年的应付总额。

三. 统计用户每月消费应付总额

- 掌握 Hive 中的日期函数
- 利用日期函数统计每个用户每个月消费的应付总额

日期获取类函数

1. 单个日期数据获取函数

Hive 提供了丰富的日期获取类函数，可以分别用于获取对应的年、月、日等单个数据，也可以获取整体数据或根据需要获取相应的数据。

在 SELECT 语句中可以通过 YEAR、MONTH、DAY、HOUR、MINUTE、SECOND 分别获取时间字符串中对应的年、月、日、时、分、秒。

语法结构

```
YEAR(STRING S)
```

- S：需要符合日期格式，可以是常量，也可以是数据表的某个字段。

Hive 从字符串中获取年、月、日、时、分、秒时有容错机制。例如月份大于 12，每满 12 会自动进位，将年份加 1，最终得到的月份的数值一定是 1～12 这个范围内的。

示例

```
SELECT YEAR('2022-05-01 12:30:45'), MONTH('2022-05-01 12:30:45'), DAY('2022-05-01 12:30:45');
SELECT HOUR('2022-05-01 12:30:45'), MINUTE('2022-05-01 12:30:45'), SECOND('2022-05-01 12:30:45');
```

这两条用常量演示这 6 个函数的取值，依次得到 2022、5、1 与 12、30、45。

```
SELECT year_month, YEAR(year_month), MONTH(year_month) FROM mmconsume_billevents LIMIT 5;
```

把同样的函数作用在账单表的 year_month 字段上，从真实数据中取出账单所属的年、月。

2. TO_DATE 函数 & WEEKOFYEAR 函数

语法结构

TO_DATE 函数可以用于从日期格式的字符串中获取日期部分。

```
TO_DATE(STRING S)
```

示例

```
SELECT TO_DATE('2022-05-01 12:30:45');
```

TO_DATE 丢掉时间部分只保留日期，结果为 2022-05-01;

语法结构

WEEKOFYEAR 函数可以用于返回日期在本年中的周数。

```
WEEKOFYEAR(STRING S)
```

示例

```
SELECT WEEKOFYEAR('2022-05-01 12:30:45');
SELECT TO_DATE(year_month), WEEKOFYEAR(year_month) FROM mmconsume_billevents LIMIT
5;
```

WEEKOFYEAR 返回日期在本年中所处的周数。直接从账单表取值，同一个月的账单会得到相同的日期与周数。

> *ISO-8601 的两条规则：*
>
> 1. *每周从周一开始（周日为第 7 天）。*
> 2. *某年的第 1 周，是第一个至少有 4 天落在本年的那一周。换言之，包含 1 月 1 日的那一周，只要新年的天数不足 4 天，整周就归给上一年。*

3. TRUNC 函数

TRUNC 函数可以用于截取日期格式字符串中的时间，截取之后日期会归零。

语法结构

```
TRUNC(STRING S[, STRING FMT])
```

- S：需要符合日期格式的字符串。
- FMT：截取的格式，支持的格式包括“MONTH”“MON”“MM”“YEAR”“YYYY”“YY”，分别表示按照月份及年份截取。
    - 若按照月份截取，则将日期归零为当月 1 号。
    - 若按照年份截取，则将日期归零为当年的 1 月 1 日。

示例

```
SELECT TRUNC('2022-05-10 12:30:45', 'MM');
SELECT TRUNC('2022-05-10 12:30:45', 'YYYY');
```

前两条分别按月份与年份截取，结果为 2022-05-01 与 2022-01-01，原来的时分秒被丢弃；

```
SELECT DISTINCT TRUNC(year_month, 'MM') FROM mmconsume_billevents;
```

第三条把账单表的账单时间按月归零并去重，得到 2022 年 1—7 月的月初日期。需要注意 FMT 必须写成大写，'MM' 与 'YYYY' 也可以分别写成 'MONTH' 与 'YEAR'。

日期计算类函数

1. DATEDIFF 函数

DATEDIFF 函数可以用于比较两个日期，返回结束日期与开始日期之间相隔的天数。

语法结构

```
DATEDIFF(STRING END, STRING START)
```

- END：结束日期。
- START：开始日期。
- DATEDIFF 函数并不要求结束日期晚于开始日期，若结束日期早于开始日期，则返回结果为负数。

示例

```
SELECT DATEDIFF('2022-07-01', '2022-05-01');
SELECT DATEDIFF('2022-05-01', '2022-07-01');
```

前两条的参数顺序刚好相反，结果分别为 61 与 -61，印证了“结束日期早于开始日期时结果为负数”；

以“最晚账单 - 最早账单”统计每个用户在统计周期内账单横跨的天数

```
SELECT phone_no, DATEDIFF(MAX(year_month), MIN(year_month)) AS days
FROM mmconsume_billevents
GROUP BY phone_no
LIMIT 10;
```

2. 日期增减函数

DATE_ADD 函数可以用于返回基于开始日期增加指定天数之后的日期。

语法结构

```
DATE_ADD(STRING STARTDATE, INT DAYS)
```

DATE_SUB 函数则可以用于返回基于开始日期减少指定天数之后的日期。

语法结构

```
DATE_SUB(STRING STARTDATE, INT DAYS)
```

示例

```
SELECT DATE_ADD('2022-05-01 00:00:00', 10);
SELECT DATE_SUB('2022-05-01 00:00:00', 10);
```

前两条的结果分别为 2022-05-11 与 2022-04-21，可见两函数的返回值只保留日期部分。

以账单表的账单时间为基准，分别向后、向前推 7 天。

```
SELECT year_month, DATE_ADD(year_month, 7), DATE_SUB(year_month, 7)
FROM mmconsume_billevents
LIMIT 5;
```

3. ADD_MONTHS 函数

ADD_MONTHS 函数可以用于返回基于指定日期增加指定月数之后的日期。

语法结构

```
ADD_MONTHS(STRING STARTDATE, INT MONTHS)
```

- MONTHS 可以是正数也可以是负数。
    - 若为正数则表示增加指定月数。
    - 若为负数则表示减少指定月数。

示例

```
SELECT ADD_MONTHS('2022-05-01', 2);
SELECT ADD_MONTHS('2022-05-01', -2);
SELECT ADD_MONTHS('2022-01-31', 1);
```

前三条的结果依次为 2022-07-01、2022-03-01、2022-02-28：月数为负数时向前推，且当目标月份没有对应的日期（2 月没有 31 号）时，结果自动取该月的最后一天。

把账单时间统一后移 6 个月

```
SELECT year_month, ADD_MONTHS(year_month, 6) FROM mmconsume_billevents LIMIT 5;
```

---

mark

任务描述

利用日期函数统计每个用户每个月消费的应付总额。

任务实现思路

- 根据 mmconsume_billevents 数据表进行数据统计。
- 在时间的获取过程中可以使用 Hive 的 MONTH 函数。
- 为避免出现类似将 2022 年 7 月及 2021 年 7 月的数据都作为同样的 7 月数据进行累加，在分组过程中，需要以用户编号、年份及月份 3 个值作为分组依据。

任务实现

统计用户每月消费应付总额。

```
SELECT phone_no, YEAR(year_month), MONTH(year_month), SUM(should_pay)
FROM mmconsume_billevents
GROUP BY phone_no, YEAR(year_month), MONTH(year_month);
```

用 YEAR、MONTH 分别取出年月，按“用户编号 + 年份 + 月份”三个值分组后再 SUM(should_pay)，这样不同年份的同名月份会被分别统计，不会混在一起累加。

四. 统计用户每月实际账单金额

- 掌握 Hive 中的数学函数
- 利用数学函数统计 2022 年度月均账单金额最高的 20 个用户

数学函数

1. 四则运算符

Hive 支持直接在字段值及常量之间进行四则运算。

四则运算符	含义
+	加法
-	减法
*	乘法
/	除法

2. 取整函数

将浮点数转换为整数，称为取整，Hive 提供了 3 种取整函数。

函数	含义	示例
ROUND(DOUBLE A[, INT D])	四舍五入，D 为保留的小数位数，省略 D 则取整	ROUND(3.14159, 2) → 3.14
FLOOR(DOUBLE A)	向下取整，返回不大于 A 的最大整数	FLOOR(3.7) → 3
CEIL(DOUBLE A)	向上取整，返回不小于 A 的最小整数，也可写作 CEILING	CEIL(3.2) → 4

> *注意 FLOOR 与 CEIL 的方向是朝数轴两端取整，而非朝零取整，因此 FLOOR(-3.2) 的结果为 -4，CEIL(-3.2) 的结果为 -3。*

任务描述

利用数学函数统计 2022 年度月均账单金额最高的 20 个用户。

任务实现思路

- 根据 mmconsume_billevents 数据表进行数据统计。
- 实际账单金额为应付金额减去优惠金额，即 should_pay - favour_fee。
- 账单表的粒度是「用户 + 账期 + 费用科目」，同一用户同一月份存在多条明细行，所以必须先把同一用户同一月份的多条明细合并成月账单，再对月份求平均。直接对明细行取 AVG(should_pay - favour_fee)，得到的是单笔明细的均值，而非月账单均值。
- 本任务只按用户编号分组，因此月份数用 COUNT(DISTINCT MONTH(year_month)) 单独统

计，且只计入该用户有账单的月份。

- 年份是行级过滤条件，写在 `WHERE` 中先过滤再聚合，不要写进 `HAVING`。
- 最后用 `ROUND(..., 2)` 保留 2 位小数，按均值降序排序后取前 20 条。

任务实现

2022 年度月均账单金额最高的 20 个用户

```
SELECT phone_no,
       ROUND(SUM(should_pay - favour_fee) / COUNT(DISTINCT MONTH(year_month)), 2)
AS average
FROM mmconsume_billevents
WHERE YEAR(year_month) = 2022
GROUP BY phone_no
ORDER BY average DESC
LIMIT 20;
```

按 `phone_no` 单列分组，结果就是 20 个用户，每人一行。分子 `SUM(should_pay - favour_fee)` 是该用户 2022 年的实付总额，分母 `COUNT(DISTINCT MONTH(year_month))` 是他有账单的月份数，二者相除即月均账单金额。因为年份已由 `WHERE` 限定为 2022，统计月份数时无需再区分年份。

> *本数据集只覆盖 2022 年 1—7 月，所以「月均」是这几个月中有账单月份的均值，并非 12 个月的均值；各用户有账单的月数不同，用月均而非总额比较，才不会因为账单月数多而占优。*

> *若 `favour_fee` 存在空值，`should_pay - favour_fee` 的结果会变成 `NULL` 并被 `SUM` 忽略，此时需改写为 `should_pay - COALESCE(favour_fee, 0)`。*

五. 查询用户宽带订单的地址数据

- 掌握 Hive 中的连接查询
- 通过连接查询获取完整的宽带订单数据

JOIN 语句的使用

1. JOIN 语句的语法结构

Hive 中的连接查询分为内连接、外连接以及左半开连接。

JOIN 语句的语法结构

```
SELECT select_expr, select_expr, ...
FROM left_table_reference
[JOIN TYPE] right_table_reference
[JOIN_CONDITION]
```

- SELECT select_expr, select_expr, ...：需要查询的字段。
- left_table_reference：连接查询中的其中一个表的表名，称为左表。
- [JOIN TYPE]：连接类型对应的关键字。
- right_table_reference：右表。
- [JOIN_CONDITION]：连接条件，通常使用 ON 子句指定。

JOIN 语句的执行顺序是从左→右，一般都将大表作为右表，以节省内存。

2. 内连接

内连接只保留连接的两个表中与连接条件相匹配的数据，关键字为 INNER JOIN。

在连接查询中通常需要引入表别名，从而简化查询语句。

查询产生账单时用户的状态：以账单表 mmconsume_billevents 为左表，与用户表 mediamatch_usermsg 内连接。

```
SELECT u.phone_no, u.run_name, b.fee_code, b.year_month
FROM mmconsume_billevents AS b
INNER JOIN
mediamatch_usermsg AS u
ON b.phone_no = u.phone_no;
```

只有同时存在于账单表与用户表中的 phone_no 才会出现在结果中，因此查到的是确实产生过账单的用户状态。

3. 外连接

- 左外连接：LEFT [OUTER] JOIN
  返回左表中的所有数据，若没有符合连接条件的右表数据，则右表数据为 NULL。
- 右外连接：RIGHT [OUTER] JOIN
  返回右表中的所有数据，若没有符合连接条件的左表数据，则左表数据为 NULL。
- 全外连接：FULL [OUTER] JOIN
  包含所有左表与右表的数据，任意表的指定字段没有符合条件的值，则使用 NULL 值代替。
- 全外连接 = 左外连接 + 右外连接 - 内连接。
- 左外连接等于交换两表的右外连接，比如

```
A left out join B
B right out join A
```

查询所有用户的状态：以用户表 mediamatch_usermsg 为左表左外连接账单表。

```
SELECT u.phone_no, u.run_name, b.fee_code, b.year_month
FROM mediamatch_usermsg AS u
LEFT JOIN
mmconsume_billevents AS b
ON b.phone_no = u.phone_no;
```

左表用户全部保留，从未产生过账单的用户其 b.fee_code、b.year_month 两列显示为 NULL——这正是它与上一条内连接的区别。

4. 左半开连接

- 关键字：LEFT SEMI JOIN。
- 查询结果：结果是左表的数据，前提是数据对于右表来说，能够满足连接条件。
- 最终查询结果中并不包含任何右表的字段。
- SELECT 和 WHERE 语句都不能引用右表中的字段。
- 因为左表中的一条数据只要匹配到右表中的一条数据，就会停止扫描右表，所以左半开连接查询比内连接查询的效率高。

左半开连接查询：只取左表字段。

```
SELECT u.phone_no, u.run_name
FROM mediamatch_usermsg AS u
LEFT SEMI JOIN
mmconsume_billevents AS b
ON b.phone_no = u.phone_no;
```

左半开连接查询错误示范：SELECT 中引用了右表的 b.fee_code、b.year_month 字段。

```
SELECT u.phone_no, u.run_name, b.fee_code, b.year_month
FROM mediamatch_usermsg AS u
LEFT SEMI JOIN
mmconsume_billevents AS b
ON b.phone_no = u.phone_no;
```

这条语句会报错：左半开连接的结果集里只允许出现左表字段，右表字段既不能出现在 SELECT 中，也不能出现在 WHERE 中。

UNION ALL 关键字

1. UNION ALL 的语法结构

UNION ALL 关键字可以用于联合两个或多个 SELECT 语句的结果集，将其最终合并为一个结果集，合并过程不能消除重复行。

UNION ALL 的语法结构

```
SELECT SENTENCE1
UNION ALL
SELECT SENTENCE2
```

- SELECT SENTENCE1、SELECT SENTENCE2：完整的 SELECT 语句，作为 UNION ALL 语句的子查询。
- 每一个 UNION ALL 子查询返回的列的数量和名字必须一样，且对应的每个字段的数据的类型也必须一致。

查询所有产生订单及账单的客户的客户编号以及客户等级情况。

```
SELECT phone_no, owner_code, owner_name
FROM mmconsume_billevents
UNION ALL
SELECT phone_no, owner_code, owner_name
FROM order_index;
```

两张表的三列按顺序纵向拼接成一个结果集，重复行会被完整保留（若要去重则改用 UNION）。

任务描述

通过连接查询获取完整的宽带订单数据。

任务实现思路

- order_index 表中使用 prodname 字段记录订购产品名称。
- 宽带订单可以使用 prodname 字段模糊匹配“联合宽带”字符串进行筛选。
- 订单数据表 order_index 与用户基本数据表 mediamatch_usermsg 之间使用内连接，并将 phone_no 作为连接条件。

任务实现

查询用户宽带订单的地址数据。

```
SELECT o.phone_no, o.prodname, u.addressoj
FROM order_index o
INNER JOIN
mediamatch_usermsg u
ON o.phone_no = u.phone_no
WHERE o.prodname LIKE '%联合宽带%';
```

订单表与用户表按 phone_no 内连接，先用 LIKE '%联合宽带%' 筛出宽带订单，再从用户表取出 addressoj（地址）字段，从而补齐订单数据里没有的地址信息。

六. 抽样统计用户订购产品情况

- 掌握 Hive 中使用桶表进行抽样查询
- 利用抽样查询对用户订购产品的情况进行抽样统计

使用桶表抽样查询

1. 抽样查询的语法结构

Hive 中的抽样查询是借助桶表实现的，抽样查询使用 TABLESAMPLE 关键字。

语法结构

```
SELECT SENTENCE
TABLESAMPLE(BUCKET x OUT OF y ON field);
```

- y 必须是桶表桶数的倍数或因子。
- 根据 y 的大小决定抽样的比例。
- x 决定了从哪个桶开始抽取。

任务描述

利用抽样查询对用户订购产品的情况进行抽样统计。

任务实现思路

- 创建一个桶表 order_index_bucket，将订单数据表 order_index 中的数据按照用户编号 phone_no 分为 4 个桶。
- 向桶表中导入数据。
- 使用抽样查询，抽取其中一个桶中的数据。

任务实现

创建桶表 order_index_bucket，按 phone_no 分 4 个桶。

```
CREATE TABLE order_index_bucket(
  phone_no string,
  prodname string,
  offerid string,
  offername string,
  business_name string,
  orderno string)
CLUSTERED BY(phone_no) INTO 4 BUCKETS
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

向桶表中导入数据。

```
INSERT OVERWRITE TABLE order_index_bucket
SELECT phone_no, prodname, offerid, offername, business_name, orderno
FROM order_index;
```

抽取第 3 个桶的数据。

```
SELECT * FROM order_index_bucket
TABLESAMPLE(BUCKET 3 OUT OF 4 ON phone_no);
```

统计每个桶中数据条数。

```
SELECT COUNT(*) FROM order_index_bucket
TABLESAMPLE(BUCKET 1 OUT OF 4 ON phone_no);
```

`TABLESAMPLE(BUCKET 1 OUT OF 4 ON phone_no)` 表示从 4 个桶中抽取第 1 个桶，`COUNT(*)` 即该桶的数据条数；把 `x` 换成 3 就是抽取第 3 个桶。由于只扫描单个桶的数据，抽样查询可以避免遍历全部数据，提高查询效率。

小结

- 在查询过程中善用内置函数可以优化查询过程，满足更多的查询需求。
- 介绍了 Hive 中使用 `JOIN` 关键字进行多表连接查询，要能够根据查询场景要求，选择恰当的连接类别。
- 使用抽样查询能够在查询过程抽取一个或多个桶中的数据，获得很好的抽样效果，可以避免遍历全部数据，提高查询效率。